



Lista 2 — Sincronização e Deadlocks

Aulas 11 a 18

Sistemas Operacionais

Exercícios sobre seção crítica, semáforos, monitores, problemas clássicos de concorrência e deadlocks. Cenários de sistemas concorrentes reais como bancos, filas de processamento e alocação de recursos.

Exercício 1

Cenário: Sistema bancário com 10 caixas eletrônicos compartilhando uma conta corrente com saldo inicial de R\$ 1.000,00.

1. Implemente em C com pthreads: 10 threads representando caixas, cada uma faz 5 operações aleatórias de saque (R\$ 10-200) ou depósito (R\$ 50-500).
2. Sem sincronização, qual o problema? Demonstre com execução e saldo final incorreto.
3. Corrija com mutex pthread_mutex_lock/unlock. O saldo final deve ser igual ao inicial + depósitos - saques.
4. Adicione um semáforo contador limitando a 3 operações simultâneas (controle de concorrência real de banco de dados).
5. Extra: Implemente leitores-escritores — múltiplos leitores de saldo podem ler juntos, mas escritor (gerente ajustando taxa) precisa exclusão.

Exercício 2

Cenário: Streaming de vídeo (Netflix-style) — buffer circular com produtor (servidor) e consumidor (cliente).

1. Implemente produtor-consumidor com buffer de 10 pacotes de 1KB usando semáforos em C.
2. Produtor insere 100 pacotes (1 por 50ms). Consumidor remove (1 por 80ms). O consumidor experience rebuffering? Simule.
3. Aumente o buffer para 20 pacotes. A situação melhora? Explique o trade-off entre tamanho de buffer e latência.
4. Implemente a solução com monitores em Python (threading.Condition) para o mesmo problema.
5. Pesquise como o Netflix usa CDN e adaptive bitrate para evitar rebuffering.

Exercício 3

Cenário: 5 processos em um sistema de banco de dados compartilham 3 tipos de recurso (R1=2 instâncias, R2=3, R3=1).

1. Crie um grafo de alocação (RAG) para um cenário de deadlock com P1-R1, P2-R2, P3-R3, P4-R1, P5-R2.
2. Aplique o algoritmo do Banqueiro: estado atual $Allocated = [[1,0,1],[0,1,1],[1,1,0],[0,0,1],[1,2,0]]$, $Max = [[3,1,1],[0,2,2],[2,2,1],[1,1,1],[2,3,0]]$. Calcule Available e determine se o estado é seguro.
3. Se não for seguro, qual processo deve ser suspenso para evitar deadlock?
4. Implemente o algoritmo do Banqueiro em Python que detecte deadlock e sugira ordem de execução segura.
5. Pesquise como o MySQL InnoDB detecta deadlocks entre transações concorrentes (innodb_deadlock_detect, wait-for graph).

Exercício 4

Cenário: Jantar dos Filósofos adaptado para alocação de recursos em cloud computing.

1. 5 containers (filósofos) compartilham 5 bancos de dados (hashis). Cada container precisa de 2 DBs para processar uma transação.
2. Implemente a solução em C com pthreads e semáforos. Use a solução do garçom (semáforo contador = 4) para evitar deadlock.
3. Modifique para usar a solução assimétrica: filósofos ímpares pegam garfo esquerdo primeiro, pares pegam direito.
4. Compare as duas abordagens em termos de throughput (transações por segundo) e justiça (fairness).
5. Extra: Simule starvation de um processo — como o aging resolve? Implemente prioridade dinâmica.

Requisitos

gcc (compilador C), Python 3, Linux, Valgrind (helgrind para detectar race conditions)